Profiling

Computer Systems Performance - Spring 2026

Lecture 3

IT UNIVERSITY OF COPENHAGEN

what should we do if our program runs poorly?

- give up!

- guess/hypothesize what the problem is and (re)-write or refactor code to resolve it
  - fast and familiar
  - …and often wrong

- **measure it => find actual problem(s) => trace the source(s) => (re)-write or refactor code => measure again**

intended learning outcome

- to become a doctor!
  - …a computer doctor

- in actuality
  - how to profile programs
  - through profiling (tools) we gain insight into our programs
  - through insight we can set up experiments to locate performance problems (bottlenecks)
  - finally, resolving the bottleneck will restore / improve program performance

what is a bottleneck?

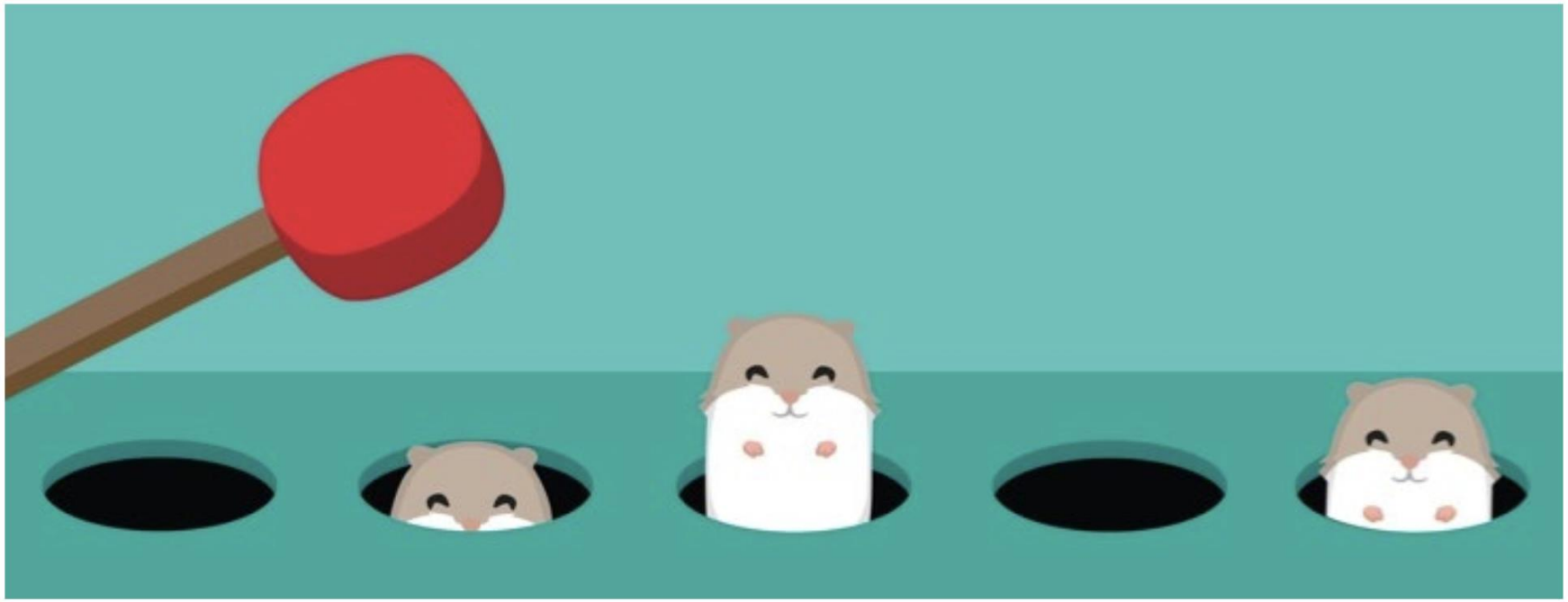
the limiting factor/component for
the capacity/throughput of a software system

it doesn’t matter if the other components

have the capacity for higher throughput

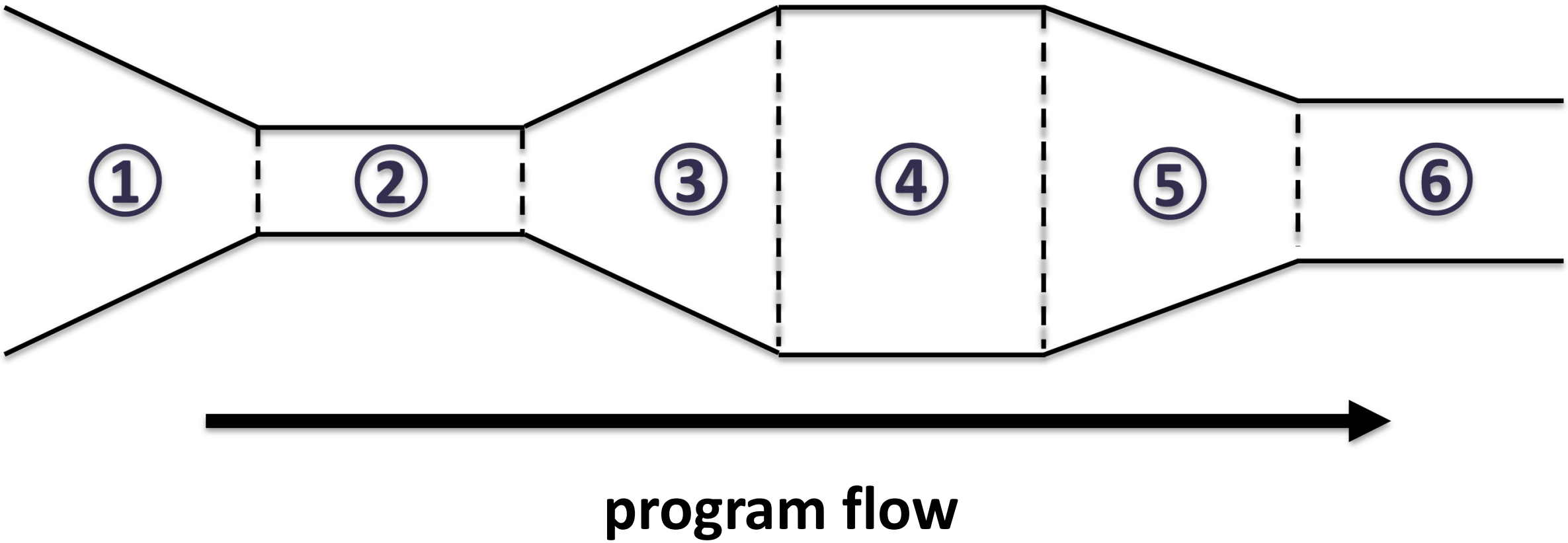
finding bottlenecks…

…is like playing whack-a-mole

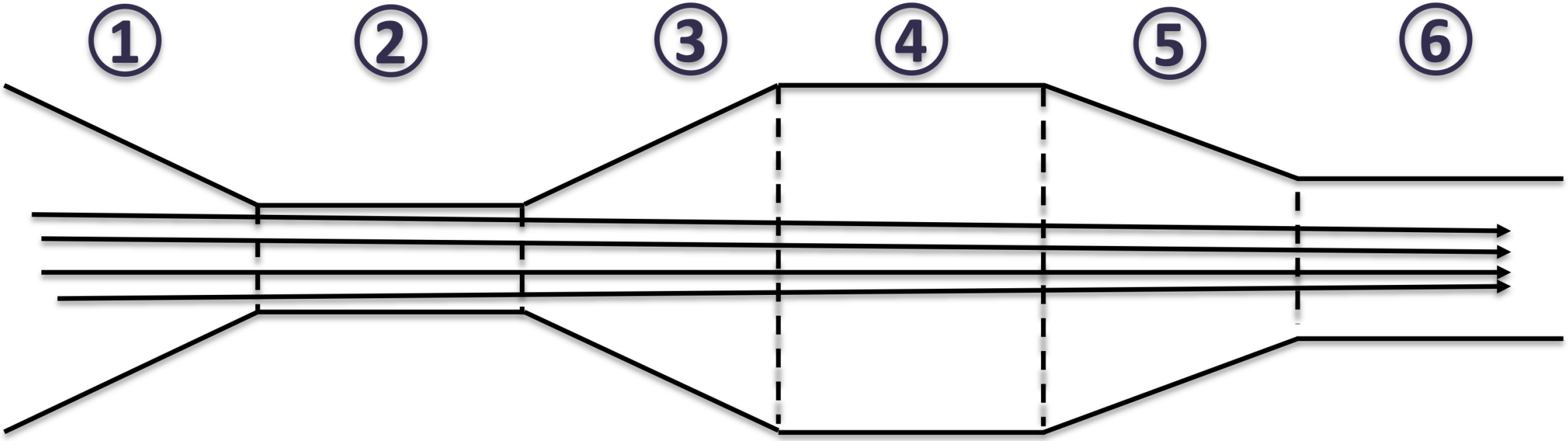


you fix one, another one pops up

finding bottlenecks…

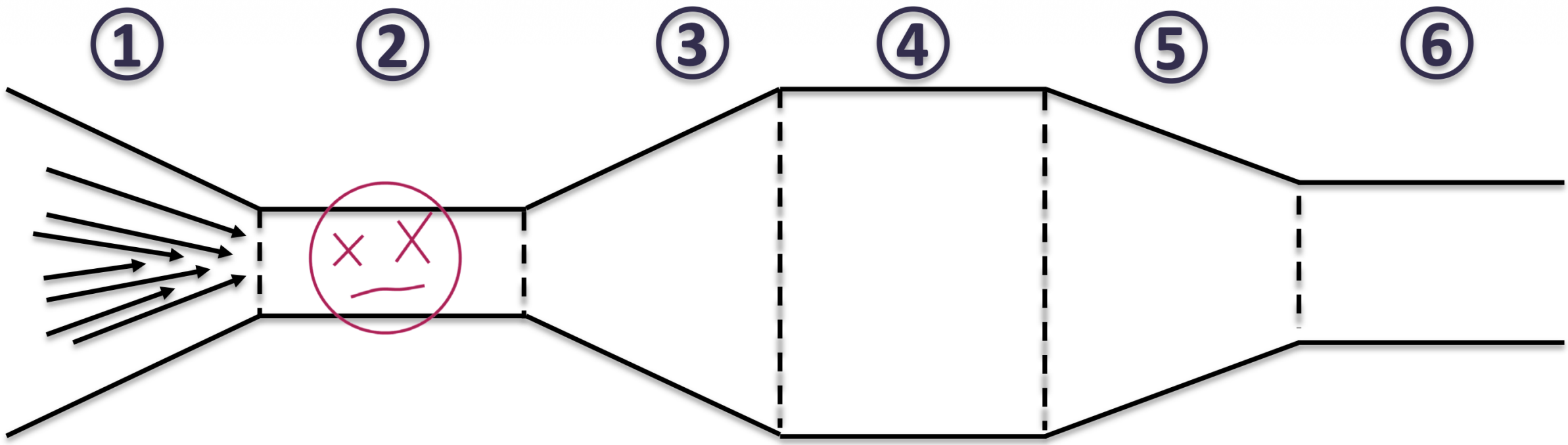


finding bottlenecks…



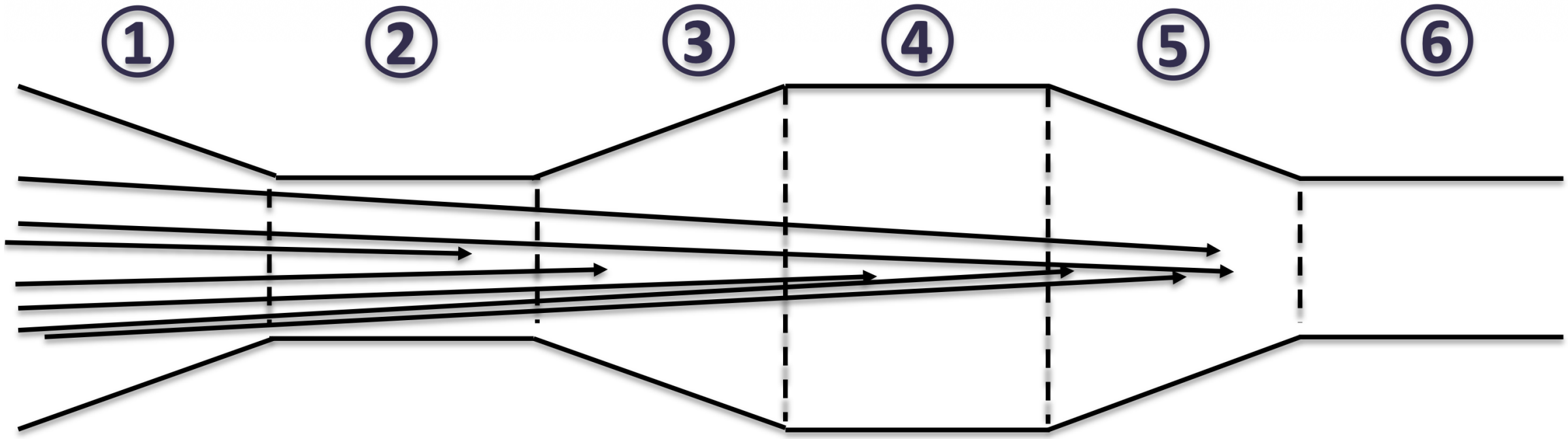
low load scenario
no bottleneck noticed

finding bottlenecks…



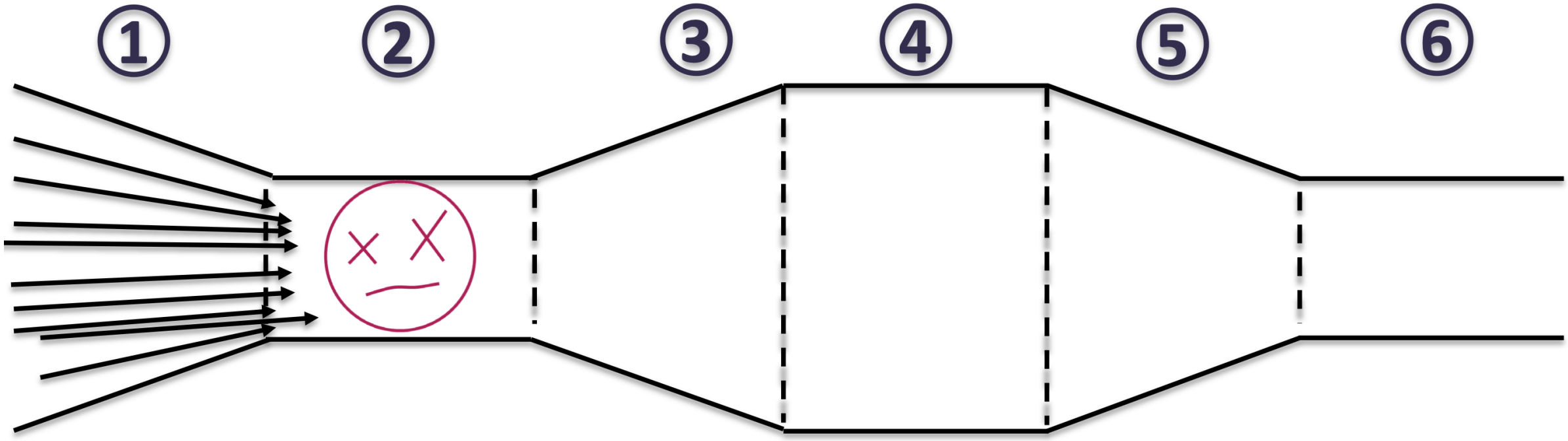
medium load scenario
things start to break
you notice something is wrong

finding bottlenecks…



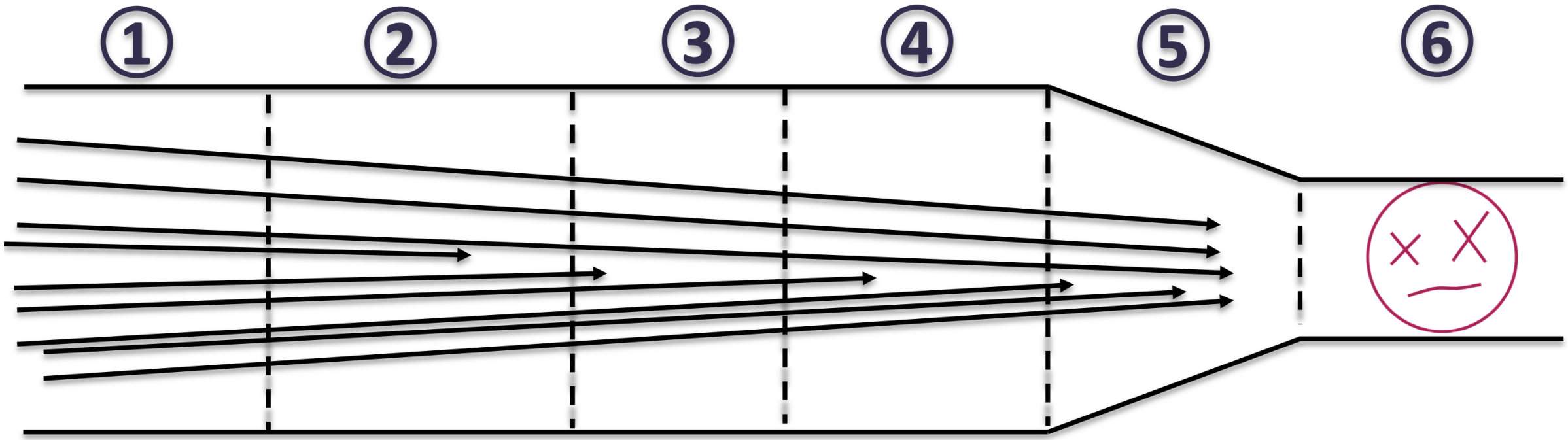
patched component #2
medium load scenario
no bottleneck noticed

finding bottlenecks…



high load scenario
things start to break again

finding bottlenecks…



patched component #2 for good
you hit the next bottleneck (#6)

and this process goes on and on …

examples of bottlenecks / solutions

- **synchronization**
  - coarse- vs fine-grained locking or
  - eliminating locks (if you can)
- **I/O (main memory, disk, network, etc.) accesses**
  - organize data and/or code to minimize this
  - better caching methods
    - in processor caches from main memory
    - in memory from local storage
    - in local storage from remote storage
- **unnecessary duplicate work**
  - sharing work across threads / functions

why/when do we try find bottlenecks?

(1) system cannot satisfy the load for an application

=> want to get higher throughput or lower latency

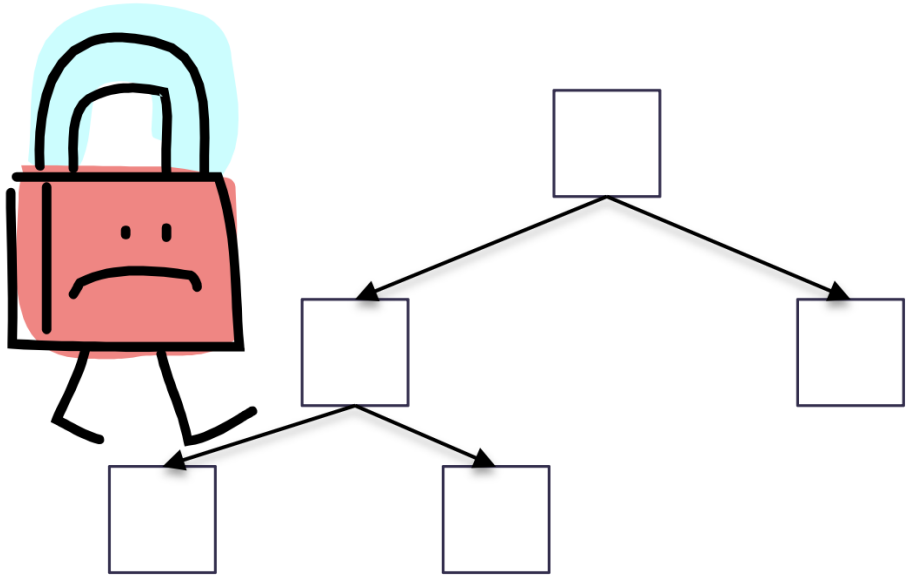
- **option 1:** throw more hardware to the problem
  ok if you have too much money (e.g., buying lots of SSDs/servers), but ***not the right solution in the long run***
- **option 2:** find & fix bottlenecks to use fewer servers to satisfy the load of an application
  better and less costly long-term solution

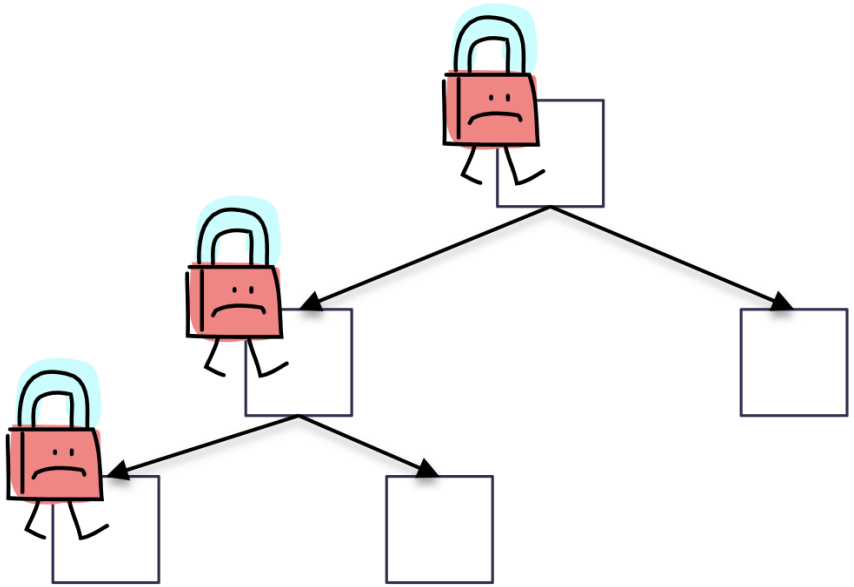
(2) want to handle the same load with less hardware to cut down costs

otherwise, no one from the industry typically makes an effort to find bottlenecks

example: coarse- vs fine-grained locking

which one is better?





easier to manage code
higher risk of having a
bottleneck when load is high

complicates code,
increases chances of bugs
allows more requests at a time

better option depends on the use case in practice

example: open vs closed data formats

which one is better?

closed relational database format from a commercial vendor

closed format may be faster to read/write, but it is also less portable if you want to switch from one data management system to another

customers may have other priorities than access latency

in real-world software development

- production cycles are typically too fast to detect all the bottlenecks before official code releases
- code simplicity may be preferred over complex code that may prevent bottlenecks in the future
- customers may have higher priorities than latency/throughput (e.g., ease of use)
- therefore, very obvious bottlenecks may exist in widely deployed commercial systems

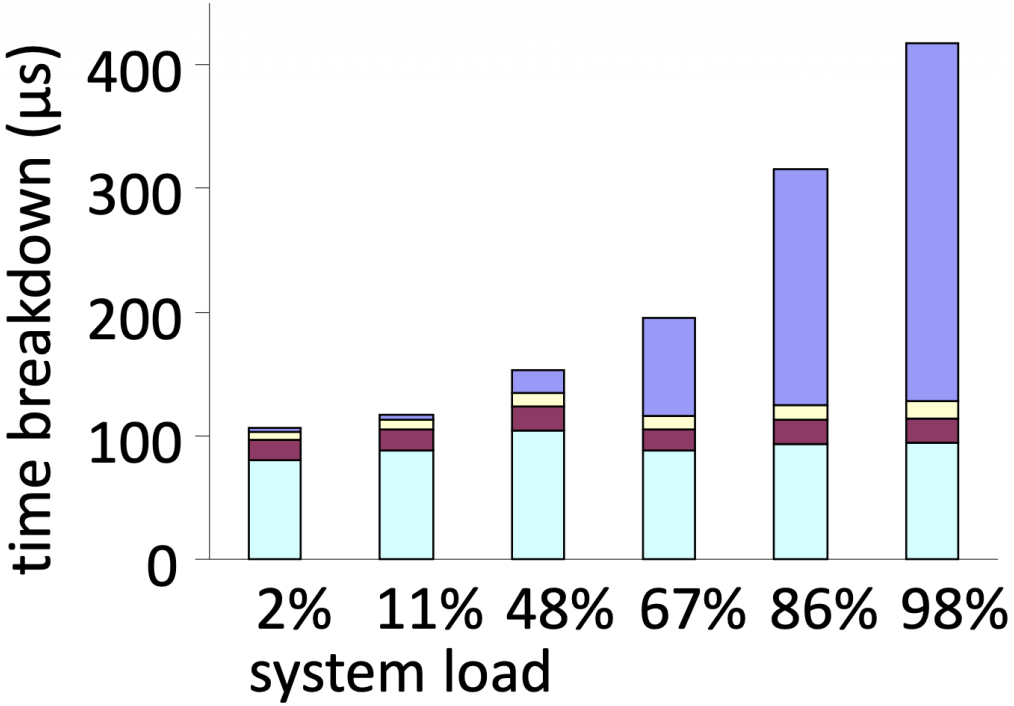
how to find bottlenecks?

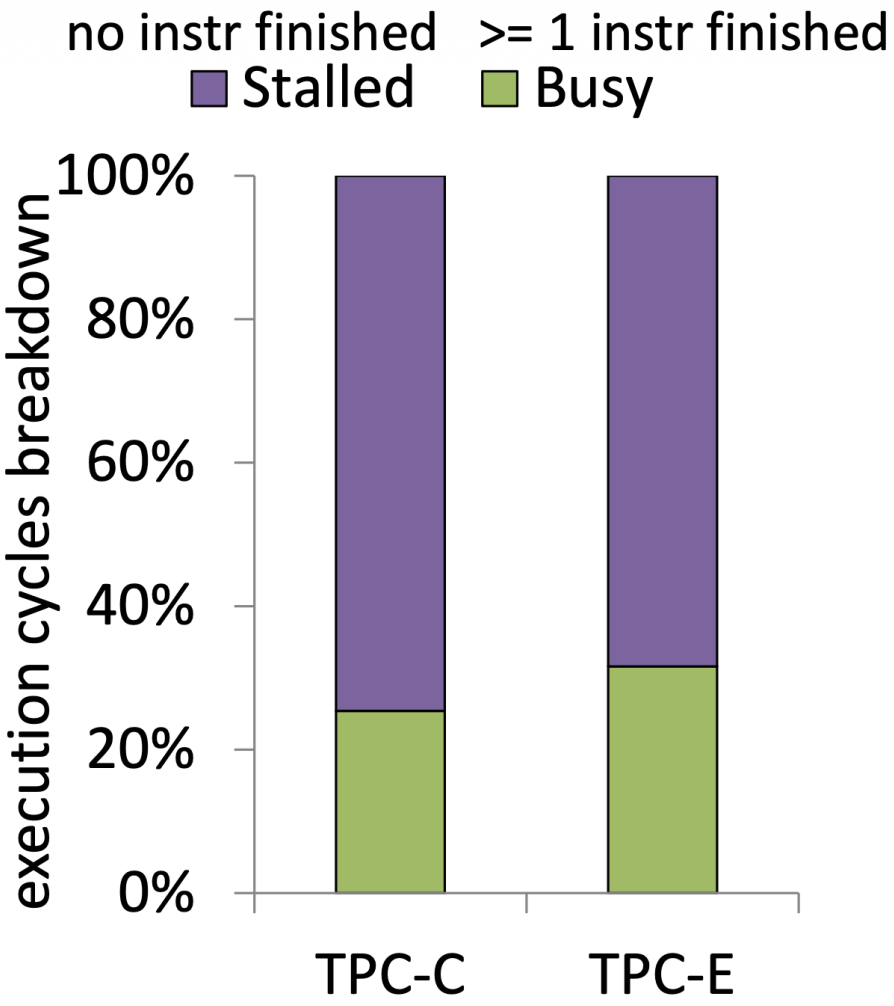
profiling

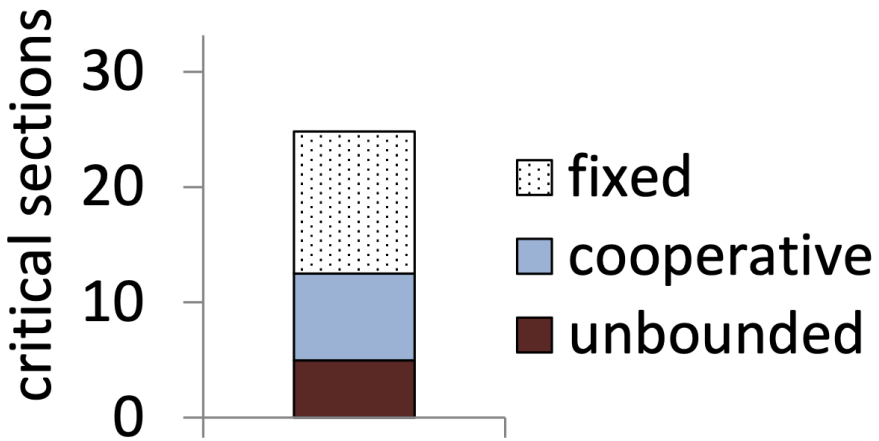
- using existing tools
  - Intel VTune, gprof, dtrace, perf,

- create your own mechanisms
  - using hardware counters
  - cpp start/stop time methods
  - collecting statistics (critical sections, page read/writes, …)

visualizing profiling output







perf

- perf is a performance analysis tool for Linux
- can report both hardware level and software level features in detail
- low overhead

# manuals
$ man perf
$ man perf record
$ man perf report

# list of events you can monitor
$ perf list

events that can be tracked

- IPC: instructions, cycles
- Branch Prediction: branches, branch-misses
- L1 Instruction Cache: L1-icache-load-misses
- L1 Data Cache: L1-dcache-loads, L1-dcache-load-misses
- LLC Data Cache: LLC-loads, LLC-load-misses
- Context Switches: cs / context-switches
- CPU Migration: cpu-migration

- many more… perf list

profiling with perf

- let’s try it out

```
$ perf stat ls
$ perf stat -d ls

$ perf stat -e L1-dcache-loads,L1-dcache-load-misses ls
```

profiling with perf - fibonacci

fib.c

```c
#include <stdio.h>
#include <stdlib.h>

u_int64_t fib(u_int64_t x) {
  if (x == 0) return 0;
  else if (x == 1) return 1;
  return fib(x-1) + fib(x-2);
}

int main (int argc, char* argv[]) {
  if(argc < 2) {
    printf("argument missing.");
  } else {
    printf("%lu\n", fib(atol(argv[1])));
  }
  return 0;
}
```

fib2.c

```c
#include <stdio.h>
#include <stdlib.h>

u_int64_t fib(u_int64_t n)
{
  u_int64_t f[n+1];
  f[0] = 0;
  f[1] = 1;
  for (u_int64_t i = 2; i <= n; i++) {
    f[i] = f[i-1] + f[i-2];
  }

  return f[n];
}

int main (int argc, char* argv[]) {
  if(argc < 2) {
    printf("argument missing.");
  } else {
    printf("%lu\n", fib(atol(argv[1])));
  }
  return 0;
}
```

fib3.c

```c
#include <stdio.h>
#include <stdlib.h>

u_int64_t fib(u_int64_t n)
{
  if(n == 0) return 0;
  else {
    u_int64_t prev = 0;
    u_int64_t current = 1;
    u_int64_t temp;
    for (u_int64_t i = 2; i <= n; i++) {
      temp = prev + current;
      prev = current;
      current = temp;
    }
    return current;
  }
}

int main (int argc, char* argv[]) {
  if(argc < 2) {
    printf("argument missing.");
  } else {
    printf("%lu\n", fib(atol(argv[1])));
  }
  return 0;
}
```

profiling with perf - fibonacci

```
# compilation
$ g++ fib.c -o fib
```

```
$ g++ fib2.c -o fib2
$ g++ fib3.c -o fib3
```

```
# basic timing
$ time ./fib 42

# perf timing % – without call graph
$ perf record ./fib 42
$ perf report

# perf timing % – with call graph
$ perf record -g ./fib 42
$ perf report

# micro-architectural events
$ perf stat -e cycles,instructions,L1-icache-load-misses,L1-dcache-load-misses,LLC-load-misses ./fib 42
```

visualizing perf reports with FlameGraph

```
$ git clone https://github.com/brendangregg/FlameGraph
$ cd FlameGraph
$ perf record -g ../fib 42
$ perf script | ./stackcollapse-perf.pl > out.perf-folded
$ cat out.perf-folded | ./flamegraph.pl > perf-fib.svg
```

FlameGraphs show where time is spent (width of boxes) and what is executing on CPU (top boxes)

profiling with perf - fibonacci

memory impact

$ perf stat -e cycles,instructions,L1-icache-load-misses,L1-dcache-load-misses,LLC-load-misses ./fib2 4200000

$ perf stat -e cycles,instructions,L1-icache-load-misses,L1-dcache-load-misses,LLC-load-misses ./fib3 4200000

# for permissions that enable perf to collect stats
# add below lines to /etc/sysctl.conf as sudo (if you want to try-out locally)
kernel.perf_event_paranoid=1
kernel.kptr_restrict=0

in general, useful site for performance analysis with perf, dtrace, flame graphs ...
http://www.brendangregg.com/overview.html

profiling with perf - core affinity

- recall exercise 2
- objective: monitor context-switches and cpu-migration

- Danyal will be demoing this part…

$ perf stat -e cpu-migrations:k,cs:k -r 5 -o no-affinity.log timeout 2s ./no-affinity

$ perf stat -e cpu-migrations:k,cs:k -r 5 -o affinity.log timeout 2s ./affinity

remember

- profiling tools help us pinpoint *where* a program is slow (discovering bottlenecks)

- given the many profiling measurement options, in practice it can be helpful to start with system monitoring tools to help decide *what* to measure (CPU, I/O, memory, network)